BEGINNER'S HANDBOOK

Python & VS Code

A complete, zero-to-hero setup guide. Install Python correctly, connect it to VS Code, and run your first script — explained step by step for absolute beginners.

Windows macOS Linux

Table of Contents

1. Understanding the Big Picture

Before installing anything, it helps to know what each piece does.

Piece	What it is	Why you need it
Python	The programming language and the interpreter that runs your code	Nothing runs without it — it's the engine
pip	Python's package installer, bundled with Python	Lets you install libraries other people wrote (e.g. `requests`, `pandas`)
VS Code	A code editor	Where you write and run your `.py` files, with helpful tools built in
Python extension	A VS Code add-on	Teaches VS Code how to run, debug, and understand Python code
Virtual environment	An isolated, project-specific copy of Python	Keeps each project's packages separate so they don't conflict

> **Think of it like this:**
> Python is the car engine. pip is how you bolt on extra parts. VS Code is the workshop you build the car in. A virtual environment is your own private garage bay, so what you install for one project doesn't spill over into another.

The order we'll set things up in: **Python → VS Code → Python extension → first script → virtual environment**. Each step takes just a few minutes.

2. Installing Python

WINDOWS

1. Go to `python.org/downloads`. Click the yellow **Download Python** button — it picks the latest version for Windows automatically.
2. Run the installer. On the very first screen, **check the box "Add python.exe to PATH"** at the bottom before clicking Install. This is the step people most often miss.
3. Click **Install Now** and wait for it to finish.
4. Open a new Command Prompt or PowerShell window and verify:

```
python --version
pip --version
```

You should see something like `Python 3.12.4` and a matching pip version.

MACOS

macOS ships with an old system Python you should avoid using directly. Install a fresh version instead.

1. *(Recommended)* Install Homebrew if you don't already have it, then install Python through it:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
brew install python
```

2. Or, download the installer directly from `python.org/downloads` and run it, clicking through the default options.
3. Open Terminal and verify:

```
python3 --version
pip3 --version
```

> **Note the "3"**
> On macOS and Linux, the commands are usually `python3` and `pip3`, since `python` may point to an old Python 2 install or nothing at all.

LINUX

1. Most distributions already include Python 3. Check first:

```
python3 --version
```

2. If it's missing or outdated, install it:

```
# Debian / Ubuntu
sudo apt update && sudo apt install python3 python3-pip python3-venv -y

# Fedora
sudo dnf install python3 python3-pip -y

# Arch
sudo pacman -S python python-pip
```

3. Installing VS Code

1. Go to `code.visualstudio.com` and click the big **Download** button — it detects your operating system automatically.
2. Run the installer.
   - **Windows:** check the boxes for "Add to PATH" and "Add 'Open with Code' action" during setup.
   - **macOS:** drag VS Code into the Applications folder, then open it once from there.
   - **Linux:** install the `.deb` / `.rpm` package, or use `sudo snap install code --classic`.
3. Skip ahead if you already installed VS Code for another guide — you can reuse the same install.

4. Setting Up VS Code for Python

Install the Python extension

1. Open VS Code. Click the Extensions icon in the left sidebar (four squares), or press `Ctrl/Cmd + Shift + X`.
2. Search for **"Python"** — the official one published by **Microsoft**. Click **Install**.

This single extension brings in IntelliSense (autocomplete), linting, debugging, and Jupyter notebook support.

Point VS Code at the right Python

1. Open any folder as your workspace: `File → Open Folder`.
2. Press `Ctrl/Cmd + Shift + P` to open the Command Palette, type **"Python: Select Interpreter"**, and press Enter.
3. Choose the Python version you installed (it usually shows the version number and install path). VS Code remembers this per folder.

> **How to tell it worked**
> The bottom-right corner of the VS Code window should now show your chosen Python version, e.g. "Python 3.12.4".

5. Running Your First Script

1. In VS Code, create a new file: `File → New File`, save it as `hello.py`.

2. Type this into the file:

```
print("Hello, world!")
name = input("What's your name? ")
print(f"Nice to meet you, {name}!")
```

3. Run it. There are three easy ways:
   - Click the ▶ **Run** arrow in the top-right corner of the editor.
   - Right-click in the file → **Run Python File in Terminal**.
   - Open the built-in terminal (`Ctrl/Cmd + \``) and type `python hello.py` (or `python3 hello.py` on macOS/Linux).

A terminal panel opens at the bottom showing the output, and waits for you to type your name.

> **Debugging**
> Click just to the left of a line number to set a red breakpoint, then press `F5` to run in debug mode — execution pauses there so you can inspect variables in the sidebar.

6. Virtual Environments & Packages

A virtual environment ("venv") is a self-contained copy of Python for one project, so its installed packages don't mix with any other project's.

Create one

Open the VS Code terminal inside your project folder and run:

```
# Windows
python -m venv .venv

# macOS / Linux
python3 -m venv .venv
```

Activate it

```
# Windows (Command Prompt)
.venv\Scripts\activate.bat

# Windows (PowerShell)
.venv\Scripts\Activate.ps1

# macOS / Linux
source .venv/bin/activate
```

Your terminal prompt now starts with `(.venv)`, showing it's active.

> **Let VS Code do it for you**
> Run "**Python: Select Interpreter**" again after creating a venv — VS Code will detect and offer `.venv` automatically, and will activate it for you in any new terminal you open.

Install packages

```
pip install requests pandas
```

Save your project's exact dependencies to a file so others (or future you) can recreate the same setup:

```
pip freeze > requirements.txt

# and later, to reinstall them all:
pip install -r requirements.txt
```

7. Everyday Reference

Command	What it does
`python --version`	Shows the installed Python version
`python file.py`	Runs a script
`python -m venv .venv`	Creates a virtual environment named `.venv`
`source .venv/bin/activate`	Activates the venv (macOS/Linux)
`deactivate`	Exits the active virtual environment
`pip install <package>`	Installs a library
`pip list`	Shows installed packages
`pip freeze > requirements.txt`	Saves exact package versions to a file
`F5`	Starts debugging the current file in VS Code
`Ctrl/Cmd + Shift + P`	Opens VS Code's Command Palette

Your first-time setup checklist

- ☐ Python installed and `python --version` works
- ☐ `pip --version` also works
- ☐ VS Code installed
- ☐ Microsoft Python extension installed
- ☐ Correct interpreter selected in VS Code
- ☐ First script written and run successfully
- ☐ Virtual environment created and activated for a real project

8. Troubleshooting

"'python' is not recognized" (Windows)

Python wasn't added to PATH during install. Re-run the installer, choose "Modify," and check "Add python.exe to PATH" — or reinstall from scratch with that box checked.

"python: command not found" (macOS/Linux)

Use `python3` instead of `python` — these systems usually don't alias `python` by default.

VS Code shows a different Python version than expected

Run **"Python: Select Interpreter"** from the Command Palette and pick the correct one — VS Code sometimes defaults to a system copy instead of your intended install or venv.

"No module named X" after pip install

Your virtual environment probably isn't activated, or you installed the package into a different environment. Check the terminal prompt for `(.venv)`, and confirm the interpreter VS Code is using matches where you ran `pip install`.

Terminal opens but doesn't auto-activate the venv

Close and reopen the integrated terminal after selecting the interpreter (`Ctrl/Cmd + \`` to close, then reopen) — VS Code applies the activation to newly opened terminals.

You now have a full Python workflow: Python runs your code, pip manages your libraries, VS Code is where you write and debug it, and virtual environments keep every project clean and separate. Happy coding!